

Homework Assignment 3: Implementation of Monitor (130 points)

Due date: See Canvas.

1 Logistics

You can work alone or in pair. However, if you choose to work alone, you can do so for a 10% bonus (I will add 10% of what you earn to the total points). Any clarifications and revisions to the assignment will be posted on canvas.

2 Sleeping Stylist with Semaphores (40 pts)

The goal of this problem is to solve an inter-thread communication problem called *Sleeping Stylist* using semaphores. There are 120 customers but only 1 stylist in a hair salon. In the salon, there are 7 chairs that customers can sit and wait for the stylist. Since there is only one stylist, only one customer can get a haircut at a time. If there are no customers in the salon, the stylist sleeps. When a customer enters the salon, the customer wakes up the stylist and then waits for the stylist to get ready. The stylist then takes the customer. If the stylist is busy with a customer and more customers arrive, they sit and wait in the 7 waiting chairs. If a customer enters the salon and it is full (there are already 7 waiting customers while a customer is getting a haircut), the customer leaves to go shopping and then comes back later to try to get a haircut again. **Every customer must eventually get a haircut (no starvation).** When the stylist finishes with a customer, she takes the next waiting customer. If there are no waiting customers, the stylist goes back to sleep. The pseudo-code is given in Figure 1. **Note that the pseudo-code only suggests a guideline. Feel free to add more parameters, variables, and functions as you think necessary.**

You will use the *pthread* package to create 120 customer threads and 1 stylist thread. The program will be called “sleepingStylistSem.c”. Use a delay loop to slow down each thread (see the pseudo-code) by adjusting the loop bound so that you get a steady stream of customers to compete to get haircuts by the stylist. You want to ensure that your program can demonstrate its operation when the salon is empty, not full and when the salon is full. Also make sure that we can see the gradual build up of customers waiting in the chairs.

You should think about the necessary debugging information that you can use to verify the correctness of your implementation. For example, what output will show that the number of waiting threads and the corresponding counter is consistent? How can you show how many customers have already got their haircuts and how many are still trying? Remember, you’ll need to convince the grader that your implementation follows our specification precisely.

You must complete your work in nuros.unl.edu. Use POSIX semaphore (*sem_init*, *sem_get*, *sem_post*, etc.). Also note that OS-X unlocks semaphores differently.

3 Sleeping Stylist with Monitor (80 pts)

The goal of this problem is to create **your own monitor** to provide synchronization support for the sleeping stylist problem. You will use the *pthread* package to create 120 customer threads and 1 stylist thread. There are 7 waiting chairs in the salon. You then need to use a semaphore to create your entry queue.

You also need to create your own Condition Variables (CVs). That is, you CANNOT USE the CV type provided by the pthread library (e.g., *pthread_cond_t*). If you use pthread CV, you will get 0 for this part. Condition variables are used to suspend processes or threads that cannot continue executing due to a specific monitor state (e.g. the stylist is busy). They are also used to awaken suspended processes or threads when the conditions are satisfiable. To create your own CVs, follow the following steps:

1. You will create a new variable type called *CV*. To do this, you will create a new structure called *cond*. The structure consists of an integer variable that indicates the number of threads blocked on a condition variable and a *semaphore* used to suspend threads. **Your implementation must follow the *signal-and-wait***

```
// ==== sleepingStylistSem.c ====//
#define CHAIRS 7
#define DELAY 1000000 // adjust this value
semaphore mutex = 1, stylist = 0 customers = 0;
int waiting = 0;

void main(void)
{
    // create a specified number of customer threads
    // and a stylist thread. Don't forget to join threads
}

void stylist(void)
{
    int j;
    while(1) {
        down(&customers);
        down(&mutex);
        waiting = waiting - 1;
        up(&stylist);
        up(&mutex);
        for (j = 0; j < DELAY; j++); // cut hair
    }
}

void customer(void)
{
    int j;
    while(1){
        down(&mutex);
        if (waiting < CHAIRS){
            waiting = waiting + 1;
            up(&customers);
            up(&mutex);
            down(&stylist);
            break;
        }
        else {
            up(&mutex);
            for (j = 0; j < DELAY; j++); // go shopping
        }
    }
}
}
```

Figure 1: Pseudo-code for the sleeping stylist problem using semaphores

discipline. If your implementation follows the *signal-and-continue* discipline, you would automatically lose 50 points. There are three operations that your CV will support. They are:

- (a) `count(cv)`—returns the number of threads blocked on the cv.
- (b) `wait(cv)`—relinquishes exclusive access to the monitor and then suspends the executing threads.
- (c) `signal(cv)`—unblocks *one* thread suspended at the head of the cv blocking queue. The signaled thread **resumes execution where it was last suspended**. The signaler *exits the monitor and suspends itself at the entry queue*.

You should pay special attention to the following questions during your implementation:

- (a) How would you guarantee that only one thread is inside the monitor at one time?
- (b) How would you make sure that a suspended thread (due to *wait*) resumes where it left off?
- (c) How would you initialize the necessary data structures to support your monitor and make them visible to all threads?
- (d) How would you produce the necessary debugging information, in addition to the required information in `mon_debugPrint`, that you can use to verify the correctness of your implementation? For example, what kind of output will show that your implementation follows the signal-and-wait discipline and not signal-and-continue? How can you show that the number of waiting threads and the corresponding counter is consistent?

2. You will create function `mon_checkCustomer` that manages the waiting list and takes a customer to the styling chair. It first invokes `signal` on condition variable `stylistAvailable` to indicate that the stylist is not busy. If the salon is empty, it then invokes `wait` on the condition variable `customerAvailable` to put the stylist to sleep.
3. You will create function `mon_checkStylist` that puts a customer on the 7 waiting chairs if the salon is not full. If the stylist is sleeping or busy, it first invokes `wait` on the condition variable `stylistAvailable`. It also invokes `signal` on condition variable `customerAvailable` to indicate that a customer is waiting.
4. You will create function `mon_debugPrint` to expose internal states of the monitor to help with debugging. The specific format of this function is provided in the pseudo-code in Figure 3.
5. You will create function `customer` and `stylist`.
6. Make sure that your program produces an output that clearly shows that it follows the **signal-and-wait discipline**. As an example, this can be done via a simple output statement that shows where each blocked thread resumes its execution to after it is awoken from the a queue.

Note that the pseudo-code given in Figures 2 and 3 provides a guideline on how to program your monitor. However, this pseudo-code may have incorrect logic. It is part of your responsibility to identify faults and implement a correctly working monitor.

These functions and your CVs will be implemented in a separate C file. Thus, you should at least have two C source files, `monitor.c` and `sleepingStylistMon.c`. You can compile `monitor.c` using `-c` flag (e.g. `gcc -c monitor.c`). This will give you the object file (`monitor.o`) that can be linked to your `sleepingStylistMon.c` (`gcc sleepingStylistMon.c monitor.o`). **You must also complete this part in *csce.unl.edu*.** You must also create a **Makefile so that we can automatically compile your code and remove the binaries using `make` and `make clean` commands, respectively.**

```
// ==== sleepingStylistMon.c ====//
#define DELAY 1000000 // adjust this value

void main(void)
{
    // create a specified number of customer threads
    // and a stylist thread. Don't forget to join threads
}
void stylist(void)
{
    // add more variables as needed
    int j;
    while(1) {
        mon_debugPrint();
        mon_checkCustomer();
        for (j = 0; j < DELAY; j++); // cut hair
    }
}
void customer(void)
{
    // add more variables as needed
    int j;
    while(1) {
        mon_debugPrint();
        if (mon_checkStylist())
            break;
        for (j = 0; j < DELAY; j++); // go shopping
    }
}
```

Figure 2: Pseudo-code for sleeping stylist problem using a monitor

4 Submission (10 pts)

Create a zip file that has all your solutions and submit it through canvas. The step to create proper directory structure is as follows:

1. Create a directory called *lastname ha3* (replace *lastname* with the submitter's lastname). If you are working with partner(s), only one submission is needed.
2. Create sub-directories: *prob1* and *prob2* under *lastname ha3*.
3. Place your solutions in the proper directory. Provide a README file and a Makefile for each problem. **To earn 10 points, each README file should:**
 - Provide the name of every member on your team.
 - Specify the instruction on how to run your solution and what to observe to ensure that it is following the signal-and-wait discipline.
 - Document the amount of time you spent on each problem.
 - Quantify the level of challenge from 0 to 5 (5 is most difficult) for each problem.
4. Once all solutions are properly stored, zip the folder *lastname ha3* and submit the zip file through canvas. Note that canvas will only take .zip extension. If you use other means to compress your file, the system will not accept it. You can use “zip -r” command in nueros.unl.edu to zip your folder.

```

// ===== monitor.c ===== //
#define CHAIR 7
cond stylistAvailable , customerAvailable;
int customer = 0;
int stylist = 0;
// add more variables as necessary (e.g. a semaphore for entry queue)
void mon_checkCustomer()
{
    stylist = stylist + 1;
    signal(stylistAvailable); // stylist's ready to cut hair
    if (customer == 0) // do not use while here
        wait(customerAvailable);
    customer = customer - 1;
}
int mon_checkStylist()
// This function may have faults.
// If you think it does, you'll need to fix it.
{
    int status = 0;
    if (customer < CHAIRS){
        customer = customer + 1;
        signal(customerAvailable);
        if (stylist == 0) // do not use while here
            wait(stylistAvailable);
        stylist = stylist - 1;
        status = 1;
    }
    return status;
}
void mon_debugPrint(void) // print how many customers are waiting
{
    // print the state of the waiting chair using the following
    // format (the following four lines are the sample output):
    //
    // |1|1|1|0|0|0|0| => 3
    // Given haircuts = X
    // Salon full = Y times
    // Salon empty = Z times
    //
    // Below is the explanation of each line.
    //
    // Line 1: For each slot, 0 indicates that the chair is
    // available, and 1 indicates that the chair is occupied.
    //
    // The value after => indicates the number of occupied
    // chairs.
    //
    // Line 2: X = number of completed haircuts.
    //
    // Line 3: Y = the number of times that all waiting chairs are
    // full. That is, each time the number of occupied
    // chairs increases to 7, this counter should go up by 7.
    //
    // Line 4: Z = the number of times that the stylist goes to
    // sleep. That is, each time the stylist goes to sleep,
    // this counter should go up by 1.
}

```

Figure 3: Pseudo-code of the monitor functions